CS 260

Big O Pt 2

Big O Review

- $2N^5 + 3N^4 + 6N^2$
- $N^5 * N^4 + N^3 - N$
- $(N^2)^3$
- $((N^7)*N)/(N^3)$

Search and Add Complexity Unsorted Data

- Arrays
  - O(n) search
  - O(1)+ add
  - O(1)+ remove
- Linked lists
  - O(n) search
  - O(1) add
  - O(1) remove

Singly Linked List vs Dynamic Array

- **Big O analysis**
  - Bag – Array-Based
    - Add
    - Delete
    - Search
  - Bag – Linked List-Based
    - Add
    - Delete
    - Search

Singly Linked List vs Dynamic Array

- **Big O analysis**
  - Stack – Array-Based
    - Push
    - Pop
    - Peek
  - Stack – Linked List-Based
    - Push
    - Pop
    - Peek

Singly Linked List vs Dynamic Array

- **Big O analysis**
  - Queue – Array-Based
    - Push
    - Pop
    - Peek
  - Queue – Linked List-Based
    - Push
    - Pop
    - Peek

Big O - Recursion

- How many times will the function be executed?
- How much time does each function call take?

```
Int exp(double x, int y) {
      If(y == 0) {
            return 1;
      } else {
            return x * exp(x, y-1);
      }
}//function
```

Execution Times & Big O

- The ratio of execution times should equal the ratio of the Big O sizes

$$\frac{{n_1}^2}{{n_2}^2} = \frac{t_1}{t_2}$$

Practical Example

- Suppose we have a O(n) algorithm
  - Ex – Sum the elements in an array
  - Suppose it takes 1 second to sum 1,000 elements
  - How long will it take to sum 2,000 elements?
  - As size doubles, execution time doubles

Linear Execution Times

- Suppose execution time is O(n)

- Suppose an insert takes 1 second in a set of 100 elements
- How long would it take to insert into 100,000 elements?

Non-Linear Execution Times

- Suppose execution time is $O(n^2)$

- Suppose a n^2 search takes .01 second to search 20 elements.
- How long would it take to search 1,000 elements?

Working With Logs

- Log n
  - Really means log base 2 of n
- Given $\log_2 1024$
  - We can solve by calculating: log 1024 / log 2
  - Another method:
  - Given $\log_2 y = x$
  - $2^x = y$

Log(n) Execution Times

- Suppose a sort is O(log n) and takes .5 seconds to search 1024 elements.
- How long would it take to search 1,048,576 elements?
  - $2^{10} = 1024$
  - $2^{20} = 1{,}048{,}576$

N(Log(n)) Execution Times

- Suppose a sort is O(n(log (n)) and takes .01 seconds to search 64 elements.
- How long would it take to search 1,024 elements?
  - $2^{6} = 64$
  - $2^{10} = 1024$

How Long Will it Take?

- Suppose an algorithm takes .01 second to handle 32 elements.
- Complete the table of execution times given the following Big O execution times.

	O(1)	O(n)	$O(n^2)$	O((log(n))	O(n(log(n)))
4					
16					
64					
256					